

Assignment 5a/b - Design Patterns

Version: February 6, 2026

Welcome to the Integrated Design Patterns Project!

In this multi-week assignment, you'll design and implement a complex simulation using two different design patterns that work together in a cohesive system. This project emphasizes not just implementing patterns, but understanding when and why to use them, and how they interact in real-world applications.

Important Reminders:

AI Use Policy: This assignment is meant for **you** to practice design patterns yourself and understand how they work in a real program. All submitted code must be authored by you and you must understand every part of it.

I will allow AI tools this time but **only** for high-level guidance (concept clarification, design discussion, debugging help, or small illustrative snippets).

You may **not** use AI to generate full solutions, large code blocks, or complete classes. You may **not** upload the assignment, requirements PDF, or project files into any AI tool to obtain a solution or step-by-step implementation.

If you use AI at all, you must disclose it in your README: list the tool, how you used it, and what parts were AI- assisted. You are fully responsible for correctness, testing, and explaining your work in your own words.

Start Early: This is a substantial assignment requiring careful planning and iteration.

Test Your Submission: Before submitting, clone your repository and ensure that your code compiles and runs correctly.

Two Deliverables: This assignment is split into two parts: Deliverable A (project setup) and Deliverable B (full implementation). This two-part structure is designed to encourage continuous work on the project.

Assignment Overview

Total Points: 100

Deliverable A (15 points): Project infrastructure setup - Git repository, Gradle configuration, initial testing setup, and design planning.

Deliverable B (85 points): Complete implementation with two integrated design patterns, comprehensive testing, documentation, and demonstration.

Note: Before starting Deliverable A, it's recommended to read through the Deliverable B requirements to understand the final goal of the project.

DELIVERABLE A: Project Setup (15 points)

Goal: Establish your project infrastructure and demonstrate that you can build, test, and plan your design.

Requirements for Deliverable A

1. Private GitHub Repository (Required - no separate points):

- Create a private GitHub repository for your project
- Add "ser316asu" as a collaborator
- All work must be done in this repository
- Implementation must be in Java

2. Git Repository Quality (3 points):

- Demonstrate proficiency with Git
- Use meaningful commit messages that explain what changed and why
- Maintain a clean commit history
- Commits should be granular (not one massive commit)

3. Gradle Configuration (3 points):

- Create a `build.gradle` file that allows the project to be built and run from command line using `gradle build` and `gradle run`
- Include Checkstyle, SpotBugs, and JUnit (4 or 5) plugins with appropriate configuration files
- **Important:** We will NOT import your project into an IDE. Everything must work from command line.

4. Checkstyle and SpotBugs Setup (2 points):

- Checkstyle and SpotBugs must be configured and executable (must run without errors). You don't need zero violations yet, but the tools must work.

5. GitHub Actions Workflow (2 points):

- Set up a basic GitHub Actions workflow that triggers on push/pull request, builds the project with Gradle, and runs Checkstyle and SpotBugs
- Workflow must pass (build succeeds, tools run successfully)

6. JUnit Test (3 points):

- Include at least one passing JUnit test that demonstrates basic functionality, runnable via `gradle test`. Can be a simple dummy test case.

7. README with Initial Design Plan (2 points):

- Create a `README.md` file in your repository root

- Include the following sections:
 - **Project Overview:** Brief description of your chosen domain (Pet Shelter)
 - **Planned Design Patterns:** Which two design patterns you plan to implement and why
 - **Integration Strategy:** High-level explanation of how you envision these patterns working together
 - **Requirements Focus:** List which 6+ functional requirements you plan to fulfill
 - **Concerns/Questions:** Any concerns or open questions about your approach
- This initial plan will be compared to your final implementation in Deliverable B to assess your design evolution

What to Submit for Deliverable A

- Submit a link to your private GitHub repository on Canvas
- Your work should be on a branch named "**DelA**"
- Merge your DelA branch to "**main**" for grading
- No PDF or screencast is required for Deliverable A

DELIVERABLE B: Full Implementation (90 points – 85 will lead to full credit)

Goal: Complete your simulation with two integrated design patterns, comprehensive testing, quality code, and thorough documentation.

Important: Deliverable B builds upon your Deliverable A setup. You should continue working in the same repository.

Requirements for Deliverable B

Work on a DelB branch.

1. Integrated Design Pattern Implementation (40 points):

- **Two Patterns:** Implement two unique GoF design patterns that work together within your application to create a cohesive and sophisticated system. The patterns should interact logically and enhance your overall design.
- **Note:** Singleton pattern is NOT permitted as it doesn't demonstrate sufficient complexity.

Pattern Integration Requirements:

- Your two patterns must interact and depend on each other in meaningful ways
- Demonstrate clear data flow or control flow between patterns
- Show how the patterns complement each other to solve complex problems

Functional Requirements:

- Demonstrate the functionality of your integrated design by fulfilling at least six requirements from the separate requirements document, you can be creative with the requirements as well.
- Your implementation should show the patterns working together, not in isolation
- Requirements can be fulfilled through pattern implementation or supporting functionality
- You **must** clearly document in your PDF/README which requirements you fulfilled and how

Code Clarity and Comments:

- Comments should explain major functionalities, complex code sections

Functional and Active Simulation:

- Your project must include a functioning simulation
- Objects must interact dynamically based on requirements

- **Data in Motion:** Create and manipulate objects over time
- **Events and Actions:** Involve events that change system state
- **Cycle-Based Logic:** Use cycles/time intervals to show progression
- **Clear Output:** Use `System.out.println` or logging to show:
 - * Current cycle/time step
 - * When key events occur
 - * Status of important objects and state transitions
 - * How design patterns are being utilized (e.g., "Factory creating new X")
- **Main must run fully automated** - we will not have time to interact during grading

2. Complete Gradle Functionality (2 points):

- Build on your Deliverable A Gradle setup to ensure all tasks work correctly: build, run, test, checkstyle, spotbugs. Project must compile and run without errors.
- **Important:** Any code that does not compile will receive 0 points

3. Code Quality (10 points):

- Adhere to coding standards as defined in your Checkstyle configuration
- Resolve SpotBugs warnings or provide justification for any that remain
- Code should be clean, readable, and well-organized

4. Complete GitHub Actions CI Pipeline (4 points):

- Build on your Deliverable A workflow to run unit tests, SpotBugs, and Checkstyle checks automatically on push/PR
- All checks must pass

5. Unit Testing with 60% Code Coverage (8 points):

- Write comprehensive JUnit tests covering your design patterns and core functionality with at least 60% code coverage (you can exclude Main, getters, and setters). Use JaCoCo or similar tool to measure coverage.
- **Documentation Requirement:** Mention your coverage in PDF or README

6. Demonstration Video (6 points):

- Record a screencast (maximum 6 minutes)
- Demonstrate the following:
 - Cloning your repository from GitHub
 - Building the project: `gradle build`
 - Running the application: `gradle run`

- Explaining how your two design patterns work together (1-2 minutes)
- Upload to YouTube, Google Drive, or similar (ensure link is accessible)

7. Complete README Documentation (4 points):

- Build on your Deliverable A README and add: link to screencast, brief explanations of each pattern **in your own words**.

8. Design Justification and Process Documentation (16 points):

- This must be included either in a separate PDF or as a detailed section in your README
- Include the following sections:

Pattern Selection Rationale (4 points):

- Explain WHY you chose each of the two design patterns
- What problem does each pattern solve in your system?
- Why are these patterns a good fit for your specific domain?

Trade-off Analysis (4 points):

- For each pattern, discuss at least one alternative pattern you considered
- Compare the pros and cons of your chosen pattern vs. the alternative
- Explain why you ultimately chose your selected pattern

Design Evolution and Process (4 points):

- Compare your initial design plan (from Deliverable A) to your final implementation
- What changed as you implemented the system? Why?
- What was the most challenging aspect of making the patterns work together?
- If you could start over, what would you do differently?
- Include any sketches, diagrams, or notes from your planning process

Resources Used (2 points):

- List resources you used to implement the Design Patterns (beyond slides and lecture videos)
- Explain how these resources influenced your implementation decisions
- Be specific about what you learned from each resource

Challenges Faced (2 points):

- Describe specific difficulties you encountered
- Explain how you overcame them
- Be honest - this shows learning and problem-solving skills!

Important Notes

Compiling Code: Your code must compile without errors. Any code that does not compile will receive 0 points. We will not debug your code. If you implemented a design pattern but cannot get it to work within the main simulation, comment it out and provide an explanation in your PDF/README. You may receive partial credit for commented-out code that demonstrates a good-faith effort.

Design Flexibility: You have flexibility in choosing which two Gang of Four design patterns you implement and how you apply them to your system. However, the patterns must work together in an integrated fashion. Make sure to clearly document your choices and implementation approach.

What to Submit for Deliverable B

1. GitHub Repository Link:

- Submit link on Canvas (also include in PDF/README)
- Your work should be on a branch named **"DelB"**
- Merge your DelB branch to **"main"** for grading

2. Screencast Link:

- Include link in your PDF/README
- Ensure link is accessible to graders

3. Documentation (PDF or detailed README):

- Submit directly on Canvas
- Must include all required sections from Design Justification
- Must include screenshots of test coverage, SpotBugs, Checkstyle reports